Property Intelligence System

Multimodal AI Platform for Automated Property Assessment,
Valuation & Renovation Intelligence

Architecture Reference Document

RealPage Builder Submission

Version 1.0

Photos · Blueprints · Inspection Forms · Legal PDFs → Structured Intelligence

Vue.js · FastAPI · SQLite · Pinecone · LangChain · LangGraph
GPT-4o · Mistral OCR · Presidio · Logfire · MCP

System Overview

What the system does and why it is built this way.

The Property Intelligence System converts messy, multimodal property data into a structured, queryable knowledge base. Property managers and underwriters upload photos, blueprints, inspection forms, and legal/reference PDFs. The system routes each input through the appropriate pipeline, stores structured results in SQLite and long-form documents in Pinecone, and exposes everything through an MCP server so an AI assistant can answer natural-language questions with cited, grounded answers.

Key Outputs

Output	Description
Condition score (1–10)	AI-assessed from photos + inspection form
Estimated value + breakdown	Deterministic formula: price/sqft × sqft × age factor
Ranked renovation needs	Priority, cost, value uplift, and ROI % per item
Recommended contractors	Matched per renovation category from SQL table
Warranty coverage answers	RAG over legal document with section citations

The Core Principle — Right Tool Per Input

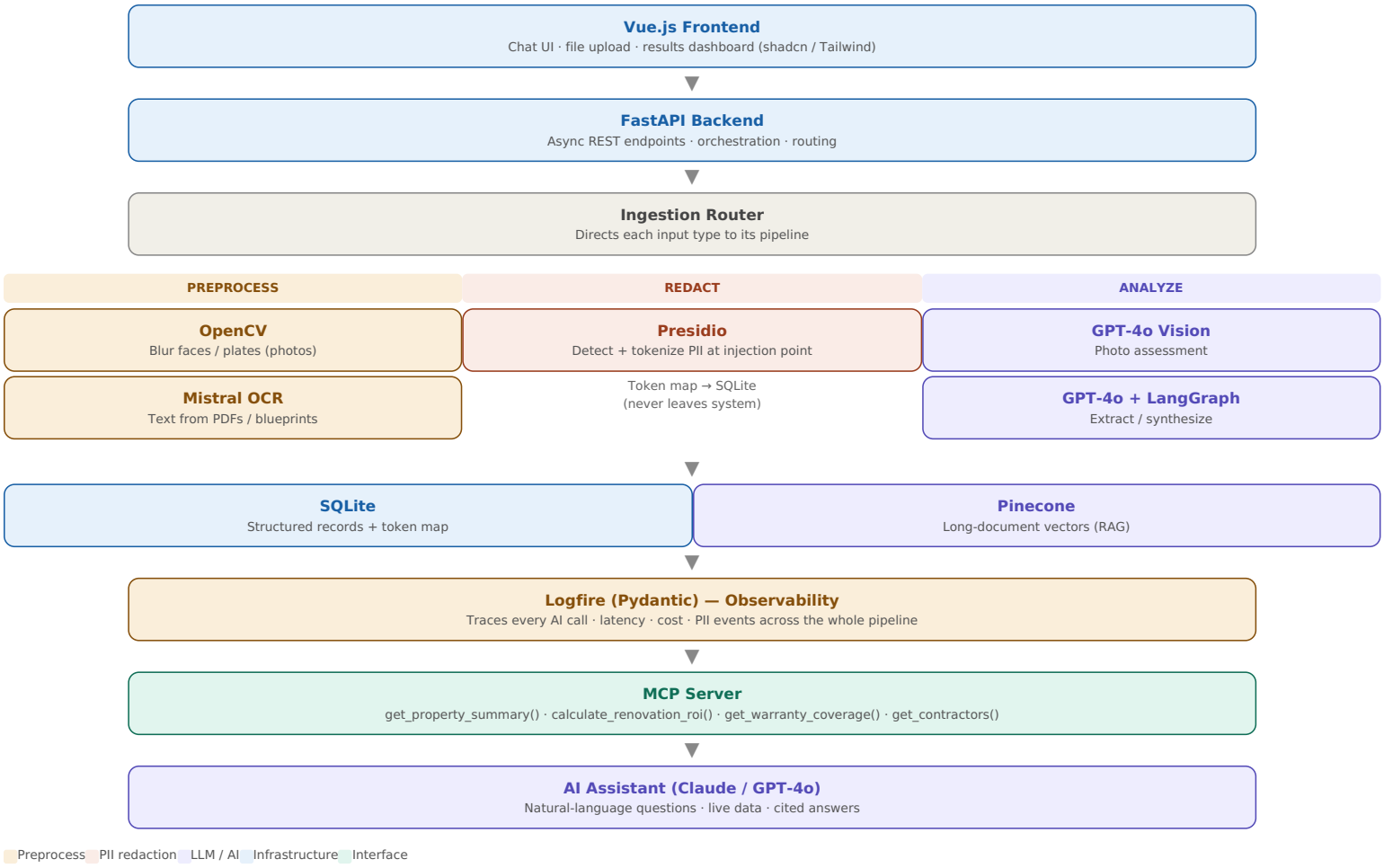
The single most important architectural decision: **do not use RAG for everything.** Each input type is matched to the right processing approach. Universal RAG makes the system slower, costlier, and more prone to hallucination.

Input	Nature	Approach	Storage
Property photos	Known fields	GPT-4o Vision → JSON	SQLite
Blueprints	Known fields	OCR + LLM extract	SQLite
Inspection form	Fixed checkbox fields	OCR + form parser	SQLite
Contractor list	Relational	Direct insert	SQLite
Valuation formulas	Deterministic math	Parse once → code	Code + MCP
Warranty / reports	Long, open-ended	RAG + chunking	Pinecone

Decision rule: *Do I know the questions people will ask, and do they map to specific fields?* Yes → SQLite. Formula → code. No + long text → RAG.

High-Level Architecture

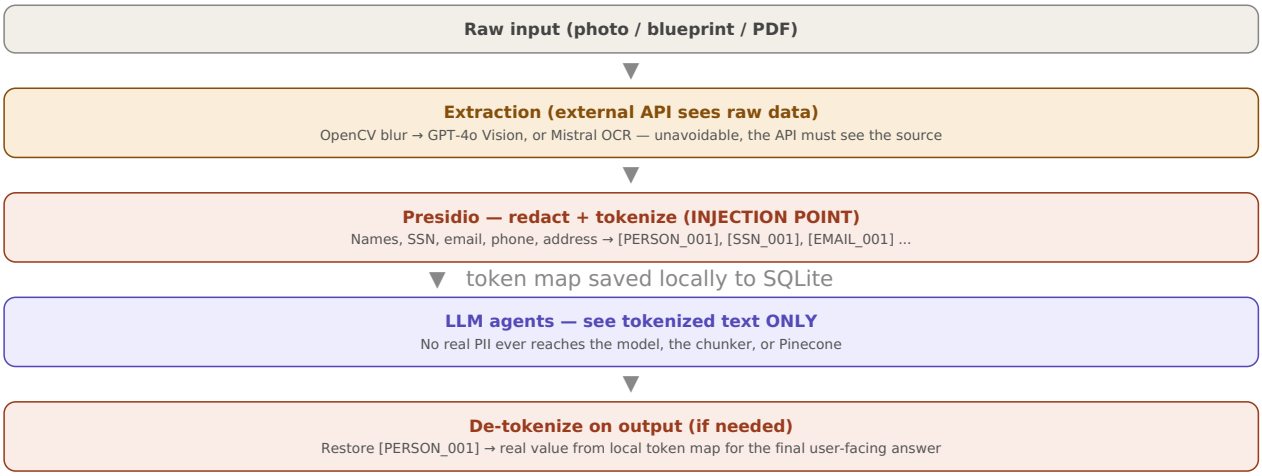
The full stack, top to bottom.



PII Redaction — The Symmetric Layer

How sensitive data is protected before it ever reaches an LLM.

Critical rule: redaction happens at the injection point — immediately after OCR or Vision extracts text, before that text touches any downstream LLM, chunker, or embedder. The window between extraction and redaction contains no storage, no logging, and no network hop.



Per-input handling

Input	External API that sees raw data	Redact after
Photos	GPT-4o Vision (+ OpenCV face/plate blur first)	Text output from Vision
Blueprints	Mistral OCR	Text output from OCR
PDF documents	Mistral OCR	Text output — highest PII risk (builder name, phone, email)

Images cannot be pre-redacted by Presidio (which works on text), so photos get an OpenCV face/license-plate blur before Vision, then their text output is redacted like everything else. The token map lives only in local SQLite and is used to reverse the tokenization when composing the final answer.

RAG Subsystem — LangChain + LangGraph

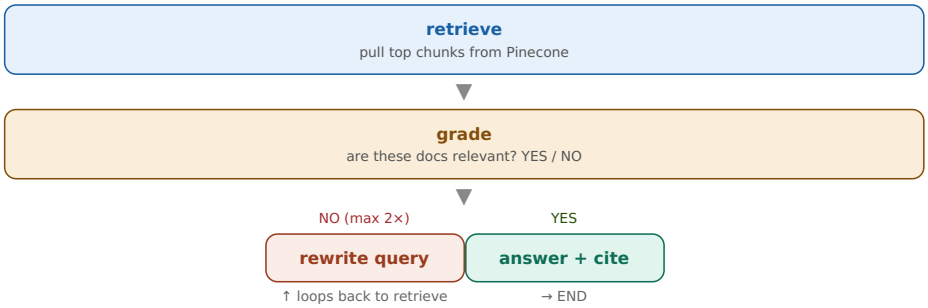
Only for long, open-ended documents: warranty, historical reports, legal contracts.

Ingestion (LangChain)

Structural chunking splits on `\n§` section markers first, so each performance standard (e.g. §7 Drywall, §28 Plumbing) stays intact as one chunk — no answer is ever split across two chunks.

Retrieval

LangGraph Agent — Multi-Step Flow



The grading + rewrite loop catches wrong retrievals before they reach the answer — essential for legal text where an incorrect answer has real consequences. Every answer cites the source section (e.g. "Section 7, 1/32 inch threshold").

Data Model & End-to-End Flow

Where each piece of information lives, and how a request flows through the system.

SQLite Tables

Table	Holds
properties	Address, sqft, year, price, condition score, estimated value
property_images	Per-image AI assessment, issues, confidence
material_assessment	Floor/wood/paint condition + source + confidence
maintenance_needs	Issue, priority, cost, value uplift, ROI, category
inspection_forms	Parsed checkbox fields, total renovation cost
documents	Doc type, file path, Pinecone namespace, key findings
renovation_companies	Contractor per category (name, site, location, phone)
pii_token_map	Token → original value map (local only)

End-to-End Request Flow

User question	Routed to	Mechanism
"What's the condition score?"	get_property_summary()	SQL lookup
"Value after roof + kitchen?"	calculate_renovation_roi()	Deterministic code
"Which contractors for the roof?"	get_contractors()	SQL lookup
"Compare Building A vs B"	get_property_summary() ×2	SQL lookup
"Is cracked drywall covered?"	get_warranty_coverage()	RAG (LangGraph)

Why This Reads as Production Thinking

- **Right tool per input** — SQL for structured data, code for formulas, RAG only for long legal docs. Not RAG-for-everything.
- **PII redacted before any external LLM** — GDPR/CCPA-aware, tokenized, reversible, local-only token map.
- **Full observability** — every AI call, cost, latency, and PII event traced in Logfire.
- **Cited, auditable answers** — warranty responses point to the exact section; grading loop prevents wrong retrievals.
- **Deterministic valuation** — numeric answers come from code, not LLM arithmetic, so they never hallucinate.

The scale pitch

At 24M+ units, RealPage's teams spend thousands of hours manually assessing properties. This pipeline turns a pile of photos, blueprints, and PDFs into structured, queryable, auditable property intelligence — instantly, at scale. That is the 10–100× leverage.